

Имитационное моделирование

Лабораторная работа №8. Реализация основных моделей в дискретно-событийном подходе

Исаева Зарина Исмаилбековна
2026-09-08

Российский университет дружбы народов имени Патриса Лумумбы

Студенческий билет: 1132232882

Москва, 2026

Содержание

1. Цель работы
2. Задание
3. Теоретическое введение
 1. Индивидуальные события
 2. Детерминированное приближение
 3. Расширения модели
4. Выполнение лабораторной работы
 1. Подготовка окружения и ядро модели
 2. Обработка результатов
 3. Базовый опыт
 4. Чувствительность к параметрам
 5. Длительность инфекции
 6. Производительность
 7. Анализ сохранённых CSV
 8. Рождения и смерти
 9. Вакцинация
 10. Латентная стадия
 11. Сопоставление с ОДУ
 12. Производные форматы
 13. Проверки модели и сборка
5. Выводы
6. Список литературы

1. Цель работы

Изучить дискретно-событийное моделирование распространения инфекции в конечной популяции. Реализовать индивидуальные процессы контактов и выздоровления на Julia, исследовать влияние параметров и расширить модель демографией, вакцинацией и латентной стадией. Сопоставить стохастический ансамбль с детерминированной моделью.

2. Задание

Работа соответствует главе 8 практикума, страницам 221–234 [1]. Выполняются базовая SIR-модель, семь дополнительных заданий §8.5 и преобразования Literate из §8.6. Для дополнительных экспериментов выбрано девять самостоятельных сценариев вместе с базовым запуском и отдельным сравнением с ОДУ. Число файлов дополнительных заданий в методичке не задано. Параметрические серии не заменяют базовый опыт.

Сценарий	Содержание	Условия
sir_des	Базовая SIR	$N=1000$, $T=40$, seed=1234
sir_des__param	Чувствительность	β , c , γ отдельно, 20 повторов
sir_duration	Фиксированная болезнь	$1/\gamma=4$, $T=80$
sir_benchmark	Производительность	$N=10000$, свежая модель
sir_csv_report	Чтение CSV	Без повторной симуляции
sir_demography	Рождения и смерти	$\Lambda=10$, $\mu=0.01$, $T=400$
sir_vaccination	Пять долей вакцинации	$t=0$, по 20 повторов
sir_seir	Латентный период	$\sigma=0.5$, $T=80$
sir_compare_ode	Сравнение с ОДУ	30 повторов, RK4

3. Теоретическое введение

3.1 Индивидуальные события

Каждый восприимчивый агент инициирует контакт после случайного интервала $\Delta t_c \sim \text{Exp}(1/c)$. В записи распределения используется среднее, поэтому интенсивность равна c . Собеседник выбирается равновероятно среди остальных живых агентов. При контакте с инфицированным заражение

происходит с вероятностью β . Время до выздоровления имеет распределение $\Delta t_r \sim \text{Exp}(1/\gamma)$. При базовых параметрах средние интервалы равны 0.1 и 4 единицам времени соответственно.

ConcurrentSim хранит очередь событий и продвигает виртуальное время до ближайшего события. Возобновляемые функции приостанавливаются на @yield timeout, а процессы регистрируются через @process [2]. Это позволяет совместить тысячи агентов без отдельного системного потока на каждого из них.

3.2 Детерминированное приближение

Для постоянной популяции $N=1000$ интенсивность заражения имеет вид $\beta c S I / (N-1)$. Знаменатель $N-1$ возникает из исключения самого агента при выборе собеседника. Соответствующая система имеет вид

$$\frac{dS}{dt} = -\frac{\beta c S I}{N-1}, \frac{dI}{dt} = \frac{\beta c S I}{N-1} - \gamma I, \frac{dR}{dt} = \gamma I.$$

Начальное эффективное число воспроизводства равно $R_{eff}(0) = \beta c S_0 / [\gamma(N-1)] \approx 1.982$. Оно характеризует раннюю тенденцию, но не предсказывает исход отдельной случайной траектории. Детерминированное приближение и конечная стохастическая популяция различаются по смыслу [3].

3.3 Расширения модели

В варианте с фиксированной болезнью ожидание равно $1/\gamma$. В SEIR заражение сначала переводит агента в E , затем после $\text{Exp}(1/\sigma)$ он становится инфекционным. Смерть планируется отдельным процессом с интенсивностью μ и конкурирует с контактом и выздоровлением. Рождения образуют независимый поток с постоянной интенсивностью Λ .

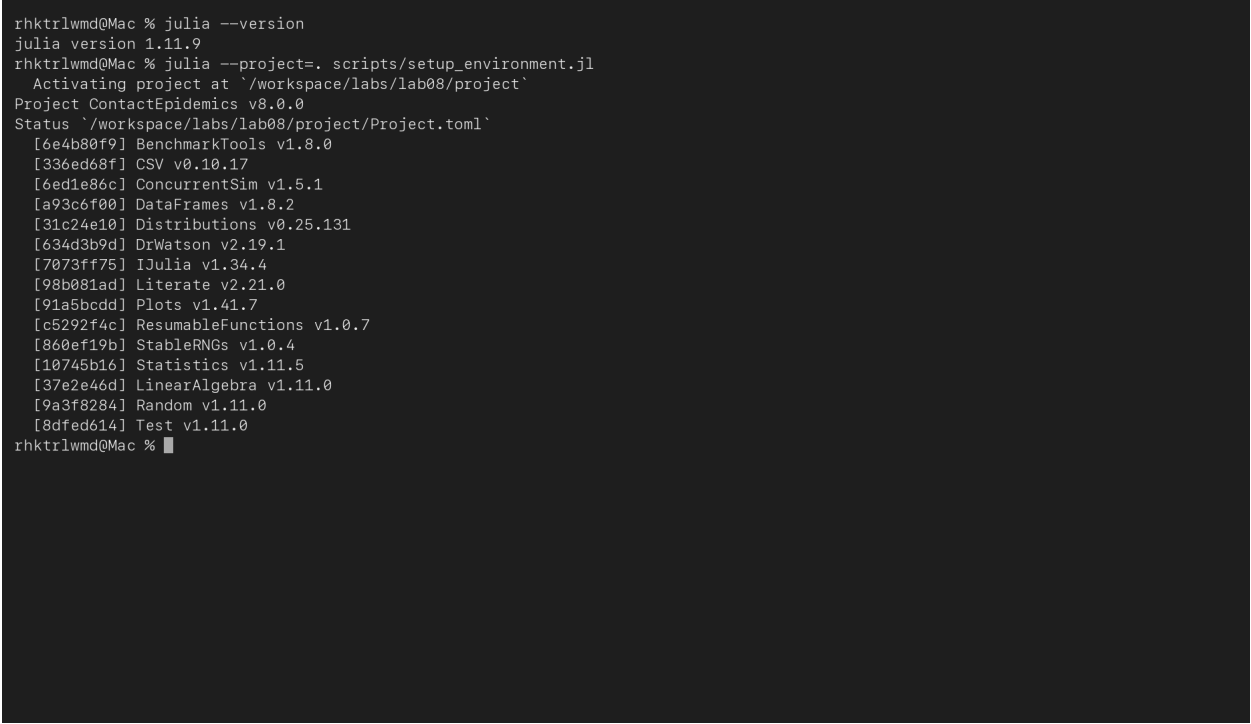
Вакцинация переводит выбранную долю текущих восприимчивых в R . Счётчик вакцинаций ведётся отдельно от числа заражений. Для идеальной вакцинации в начале опыта условие подавления раннего роста имеет вид $f > 1 - \gamma(N-1)/(\beta c S_0) \approx 0.4955$. Это порог среднего приближения, а не гарантия отсутствия отдельных заражений.

4. Выполнение лабораторной работы

4.1 Подготовка окружения

Проект ContactEpidemics использует Julia 1.11.9. Версии зависимостей закреплены в Manifest.toml. Папка src содержит ядро и обработку результатов, scripts содержит литературные исходники. Чистый код помещается в generated, выполненные блокноты в notebooks, фрагменты Quarto в markdown. Данные сохраняются в data/sims и data/analysis. Все рисунки находятся в едином каталоге image лабораторной.

```
pwd
julia --version
julia --project=. scripts/setup_environment.jl
make test
```



```
rhktrlwmd@Mac % julia --version
julia version 1.11.9
rhktrlwmd@Mac % julia --project=. scripts/setup_environment.jl
Activating project at `./workspace/labs/lab08/project`
Project ContactEpidemics v8.0.0
Status `./workspace/labs/lab08/project/Project.toml`
 [6e4b80f9] BenchmarkTools v1.8.0
 [336ed68f] CSV v0.10.17
 [6ed1e86c] ConcurrentSim v1.5.1
 [a93c6f00] DataFrames v1.8.2
 [31c24e10] Distributions v0.25.131
 [634d3b9d] DrWatson v2.19.1
 [7073ff75] IJulia v1.34.4
 [98b081ad] Literate v2.21.0
 [91a5bcdd] Plots v1.41.7
 [c5292f4c] ResumableFunctions v1.0.7
 [860ef19b] StableRNGs v1.0.4
 [10745b16] Statistics v1.11.5
 [37e2e46d] LinearAlgebra v1.11.0
 [9a3f8284] Random v1.11.0
 [8dfed614] Test v1.11.0
rhktrlwmd@Mac %
```

Рисунок 1: Проверка Julia и зависимостей

Рисунок 1 показана проверка рабочей среды перед вычислениями. Ниже приведено полное описание непосредственных зависимостей проекта.

Листинг файла project/Project.toml.

```
name = "ContactEpidemics"
uuid = "cfe7c3ac-ef99-48ac-9581-c6a51e8a9bc1"
authors = ["rhktrlwmd"]
version = "8.0.0"

[deps]
BenchmarkTools = "6e4b80f9-dd63-53aa-95a3-0cdb28fa8baf"
CSV = "336ed68f-0bac-5ca0-87d4-7b16caf5d00b"
ConcurrentSim = "6ed1e86c-fcaf-46a9-97e0-2b26a2cdb499"
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
Distributions = "31c24e10-a181-5473-b8eb-7969acd0382f"
DrWatson = "634d3b9d-ee7a-5ddf-bec9-22491ea816e1"
```

```

IJulia = "7073ff75-c697-5162-941a-fcdaad2a7d2a"
LinearAlgebra = "37e2e46d-f89d-539d-b4ee-838fcccc9c8e"
Literate = "98b081ad-f1c9-55d3-8b20-4c87d4299306"
Plots = "91a5bcdd-55d7-5caf-9e0b-520d859cae80"
Random = "9a3f8284-a2c9-5f02-9a11-845980a1fd5c"
ResumableFunctions = "c5292f4c-5179-55e1-98c5-05642aab7184"
StableRNGs = "860ef19b-820b-49d6-a774-d7a799459cd3"
Statistics = "10745b16-79ce-11e8-11f9-7d13ad32a3b2"
Test = "8dfed614-e22c-5e08-85e1-65c5234f0b40"

```

```

[compat]
CSV = "0.10"
ConcurrentSim = "1"
DataFrames = "1"
Distributions = "0.25"
DrWatson = "2"
IJulia = "1"
Literate = "2"
Plots = "1"
ResumableFunctions = "1"
StableRNGs = "1"
julia = "1.11"

```

Листинг файла `project/scripts/setup_environment.jl`.

```

using Pkg
Pkg.activate(joinpath(@__DIR__, ".."))
Pkg.instantiate()
Pkg.status()

```

4.2 Ядро модели

Состояния кодируются числами S=1, E=2, I=3, R=4, смерть обозначается 0. Список живых агентов и обратный индекс позволяют удалять умерших без линейного поиска. Единственная функция перехода одновременно меняет состояние, счётчики и журнал. После каждого ожидания проверяется текущее состояние агента. Это предотвращает заражение вакцинированного и выздоровление умершего.

Листинг файла `project/src/sir_model.jl`.

```

module ContactProcesses
using ConcurrentSim, ResumableFunctions, Distributions, Random
using StableRNGs, DataFrames
export Settings, Population, prepare, simulate, table, check_balance
export vaccinate!, transition!, activate!, finish!, sample_path

Base.@kwdef struct Settings
    beta::Float64 = 0.05
    contact::Float64 = 10.0
    recovery::Float64 = 0.25
    latent::Float64 = Inf
    fixed::Bool = false
    mortality::Float64 = 0.0
    births::Float64 = 0.0
    vaccination_time::Float64 = Inf
    vaccination_fraction::Float64 = 0.0
end

const Record = NamedTuple{(:t,:event,:person,:S,:E,:I,:R,:B,:D,:V,:cases),
    Tuple{Float64,Symbol,Int,Int,Int,Int,Int,Int,Int,Int,Int}}

mutable struct Population
    clock::Simulation
    config::Settings
    rng::StableRNG
    state::Vector{Int}

```

```

    living::Vector{Int}
    slot::Vector{Int}
    counts::Vector{Int}
    initial::Int
    born::Int
    died::Int
    vaccinated::Int
    cases::Int
    log::Vector{Record}
    active::Bool
end

function record!(m, event, id)
    s,e,i,r = m.counts
    push!(m.log, (t=Float64(now(m.clock)), event=event, person=id,
        S=s,E=e,I=i,R=r,B=m.born,D=m.died,V=m.vaccinated,cases=m.cases))
end

"""Change a living person's compartment and append one atomic ledger row."""
function transition!(m, id, destination, event)
    previous = m.state[id]
    previous == 0 && return false
    previous == destination && return false
    m.counts[previous] -= 1
    m.state[id] = destination
    if destination == 0
        position = m.slot[id]
        replacement = last(m.living)
        m.living[position] = replacement
        m.slot[replacement] = position
        pop!(m.living)
        m.slot[id] = 0
        m.died += 1
    else
        m.counts[destination] += 1
    end
    event == :infection && (m.cases += 1)
    event == :vaccination && (m.vaccinated += 1)
    record!(m, event, id)
    true
end

@resumable function disease(clock::Simulation, m::Population, id::Int)
    while m.state[id] == 1
        @yield timeout(clock, rand(m.rng, Exponential(1/m.config.contact)))
        # A pending timeout does not override death or vaccination.
        m.state[id] == 1 || return
        n = length(m.living)
        n <= 1 && continue
        draw = rand(m.rng, 1:n-1)
        draw >= m.slot[id] && (draw += 1)
        other = m.living[draw]
        if m.state[other] == 3 && rand(m.rng) < m.config.beta
            target = isfinite(m.config.latent) ? 2 : 3
            transition!(m,id,target,:infection)
        end
    end
    if m.state[id] == 2
        @yield timeout(clock,rand(m.rng,Exponential(1/m.config.latent)))
        m.state[id] == 2 || return
        transition!(m,id,3,:onset)
    end
    if m.state[id] == 3
        duration = m.config.fixed ? 1/m.config.recovery :
            rand(m.rng,Exponential(1/m.config.recovery))
        @yield timeout(clock,duration)
        m.state[id] == 3 || return
        transition!(m,id,4,:recovery)
    end
end

@resumable function death(clock::Simulation, m::Population, id::Int)
    @yield timeout(clock,rand(m.rng,Exponential(1/m.config.mortality)))
    transition!(m,id,0,:death)
end

function launch!(m,id)
    @process disease(m.clock,m,id)
end

```

```

    if m.config.mortality > 0
      @process death(m.clock,m,id)
    end
end

@resumable function immigration(clock::Simulation,m::Population)
  while true
    @yield timeout(clock,rand(m.rng,Exponential(1/m.config.births)))
    id = length(m.state)+1
    push!(m.state,1)
    push!(m.living,id)
    push!(m.slot,length(m.living))
    m.counts[1] += 1
    m.born += 1
    record!(m,:birth,id)
    launch!(m,id)
  end
end

"""Vaccinate a random fraction of current susceptibles, without new cases."""
function vaccinate!(m, fraction)
  0 <= fraction <= 1 || throw(ArgumentError("invalid fraction"))
  candidates = filter(id -> m.state[id] == 1,m.living)
  shuffle!(m.rng,candidates)
  for id in candidates[1:floor(Int,fraction*length(candidates))]
    transition!(m,id,4,:vaccination)
  end
end

@resumable function vaccination(clock::Simulation,m::Population)
  @yield timeout(clock,m.config.vaccination_time)
  vaccinate!(m,m.config.vaccination_fraction)
end

"""Build a fresh unstarted model; compartments are S=1, E=2, I=3, R=4."""
function prepare(; u0=(990,10,0), seed=1234, config=Settings())
  all(x -> x >= 0,u0) || throw(ArgumentError("negative population"))
  0 <= config.beta <= 1 || throw(ArgumentError("invalid beta"))
  config.contact > 0 && config.recovery > 0 && config.latent > 0 ||
    throw(ArgumentError("rates must be positive"))
  config.mortality >= 0 && config.births >= 0 ||
    throw(ArgumentError("negative demographic rate"))
  config.vaccination_time >= 0 || throw(ArgumentError("negative time"))
  0 <= config.vaccination_fraction <= 1 ||
    throw(ArgumentError("invalid fraction"))
  s,i,r = u0
  n = sum(u0)
  states = vcat(fill(1,s),fill(3,i),fill(4,r))
  m = Population(Simulation(),config,StableRNG(seed),states,
    collect(1:n),collect(1:n),[s,0,i,r],n,0,0,0,i,Record[],false)
  record!(m,:initial,0)
  m
end

function activate!(m)
  m.active && throw(ArgumentError("model already active"))
  m.active = true
  for id in copy(m.living)
    launch!(m,id)
  end
  m.config.births > 0 && (@process immigration(m.clock,m))
  isfinite(m.config.vaccination_time) && (@process vaccination(m.clock,m))
  m
end

function finish!(m,T)
  T > now(m.clock) || throw(ArgumentError("horizon must advance"))
  run(m.clock,Float64(T))
  record!(m,:horizon,0)
  m
end

function simulate(; T=40.0,kwargs...)
  finish!(activate!(prepare(;kwargs...)),T)
end

table(m) = DataFrame(m.log)

```



```

"""Check conservation at every event, plus the live-index bijection."""
function check_balance(m)
    all(row -> row.S+row.E+row.I+row.R == m.initial+row.B-row.D &&
        min(row.S,row.E,row.I,row.R) >= 0,m.log) &&
    sum(m.counts) == length(m.living) &&
    all(k -> m.slot[m.living[k]] == k,eachindex(m.living)) &&
    all(j -> count(==(j),m.state) == m.counts[j],1:4)
end

"""Right-continuous sampling preserves event jumps and duplicate times."""
function sample_path(data,grid; column=:I)
    [data[searchsortedlast(data.t,t),column] for t in grid]
end
end

```

4.3 Обработка результатов

Для каждой траектории сохраняются журнал событий и параметры запуска в имени CSV. Отдельно вычисляются максимум I, его время, $R(T)$, $I(T)$, накопленные инфекции и вакцинации. Ансамбль хранит метрики отдельных прогонов и среднюю траекторию отдельно. Временные ряды интерполируются справа непрерывно.

Листинг файла `project/src/StudyTools.jl`.

```

module StudyTools
using ..ContactProcesses
using CSV, DataFrames, Plots, Statistics
export save_run, run_metrics, draw_states, imagefile, save_table, ensemble
const ROOT = normpath(joinpath(@_DIR_, ".."))
const IMAGES = normpath(joinpath(ROOT, "..", "image", "results"))
imagefile(name) = (mkpath(IMAGES); joinpath(IMAGES, name*".png"))

function save_table(name, df)
    directory = joinpath(ROOT, "data", "analysis")
    mkpath(directory)
    CSV.write(joinpath(directory, name*".csv"), df)
end

function run_metrics(m)
    d = table(m)
    peak = argmax(d.I)
    (; peak=maximum(d.I), time_peak=d.t[peak], R_T=last(d.R),
       cases=last(d.cases), I_T=last(d.I), vaccinated=last(d.V),
       population=length(m.living), balance=check_balance(m))
end

function save_run(m; label="sir", seed=1234, T=40.0)
    c = m.config
    name = "$(label)_N$(m.initial)_b$(c.beta)_c$(c.contact)" *
        "_g$(c.recovery)_seed$(seed)_T$(T).csv"
    folder = joinpath(ROOT, "data", "sims")
    mkpath(folder)
    CSV.write(joinpath(folder, name), table(m))
end

function draw_states(m, name; title="SIR")
    d = table(m)
    columns = isfinite(m.config.latent) ? [:S, :E, :I, :R] : [:S, :I, :R]
    p = plot(d.t, Matrix{Float64}(d[:, columns]), label=permutedims(string.(columns)),
             xlabel="Time", ylabel="People", title=title, linewidth=2,
             seriestype=:steppost, size=(1050, 620))
    savefig(p, imagefile(name))
    p
end

"""Return separate per-run metrics and the mean path, never conflate peaks."""
function ensemble(config; repeats=20, T=80.0, u0=(990, 10, 0))
    rows = NamedTuple[]

```

```

grid = collect(0.0:0.25:T)
paths = Matrix{Float64}(undef, length(grid), repeats)
for k in 1:repeats
    m = simulate(;config,T,u0,seed=1233+k)
    push!(rows, (;replicate=k, run_metrics(m)...))
    paths[:,k] = sample_path(table(m), grid)
end
(metrics=DataFrame(rows), path=DataFrame(t=grid,
    mean_I=vec(mean(paths; dims=2)), sd_I=vec(std(paths; dims=2))))
end
end

```

Листинг файла `project/src/ContactEpidemics.jl`.

```

module ContactEpidemics
include("sir_model.jl")
include("StudyTools.jl")
end

```

4.4 Базовый опыт

Параметры §8.2: 990 восприимчивых, 10 заражённых, горизонт 40.

```
julia --project=. scripts/sir_des.jl
```

4.4.1 Литературный исходник

Параметры §8.2: 990 восприимчивых, 10 заражённых, горизонт 40.

```

using DrWatson
@quickactivate "ContactEpidemics"
using ContactEpidemics.ContactProcesses
using ContactEpidemics.StudyTools
using DataFrames, CSV, Plots, Statistics

```

```

m = simulate()
@assert check_balance(m)
save_run(m)
display(DataFrame([run_metrics(m)]))

```

График показывает состояния после каждого события.

```

figure = draw_states(m, "baseline"; title="SIR: beta=0.05, c=10, gamma=0.25")
display(plot(figure; size=(780, 420)))

```

This page was generated using [Literate.jl](#).

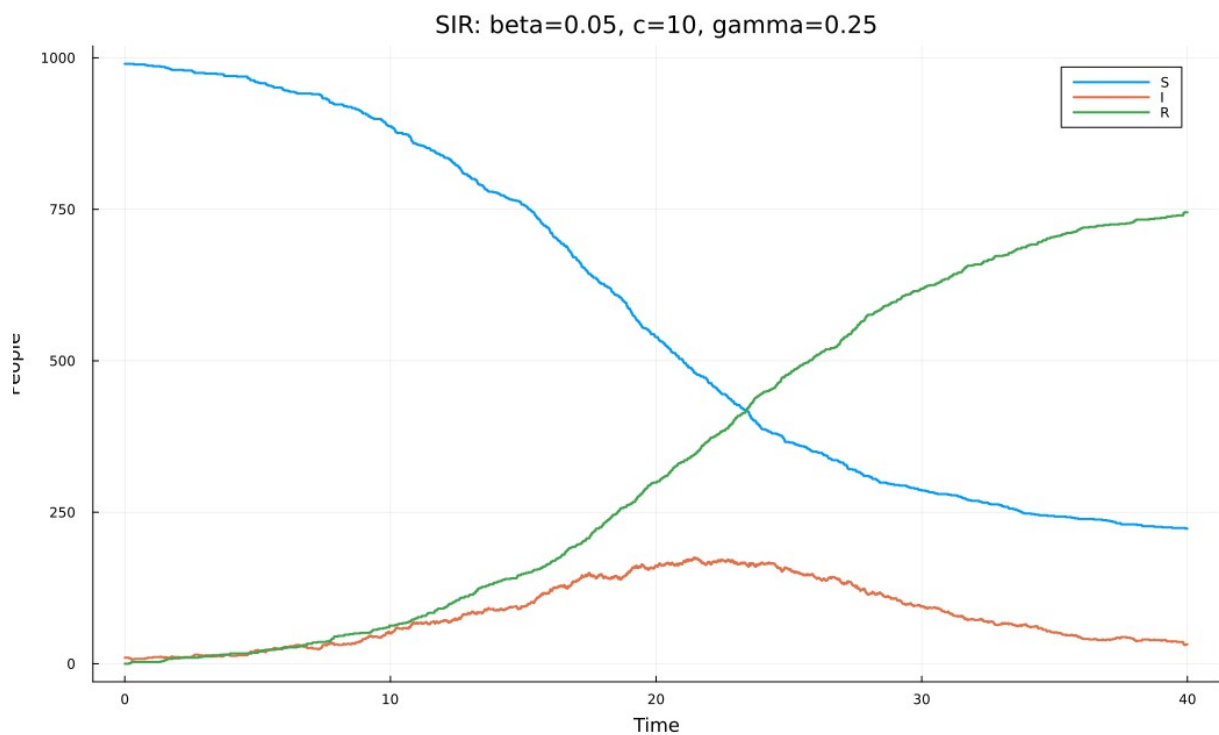


Рисунок 2: Базовый опыт: результаты расчёта

В базовом опыте пик I равен 175 при $t=21.4162$. К горизонту 40 получено $S=223$, $I=32$, $R=745$; накопленное число инфицированных с учётом начальных десяти равно 777. Базовый опыт использует исходные параметры без изменения. $R(40)$ описывает число выздоровевших к горизонту; при $I(40)>0$ это не окончательный размер эпидемии.

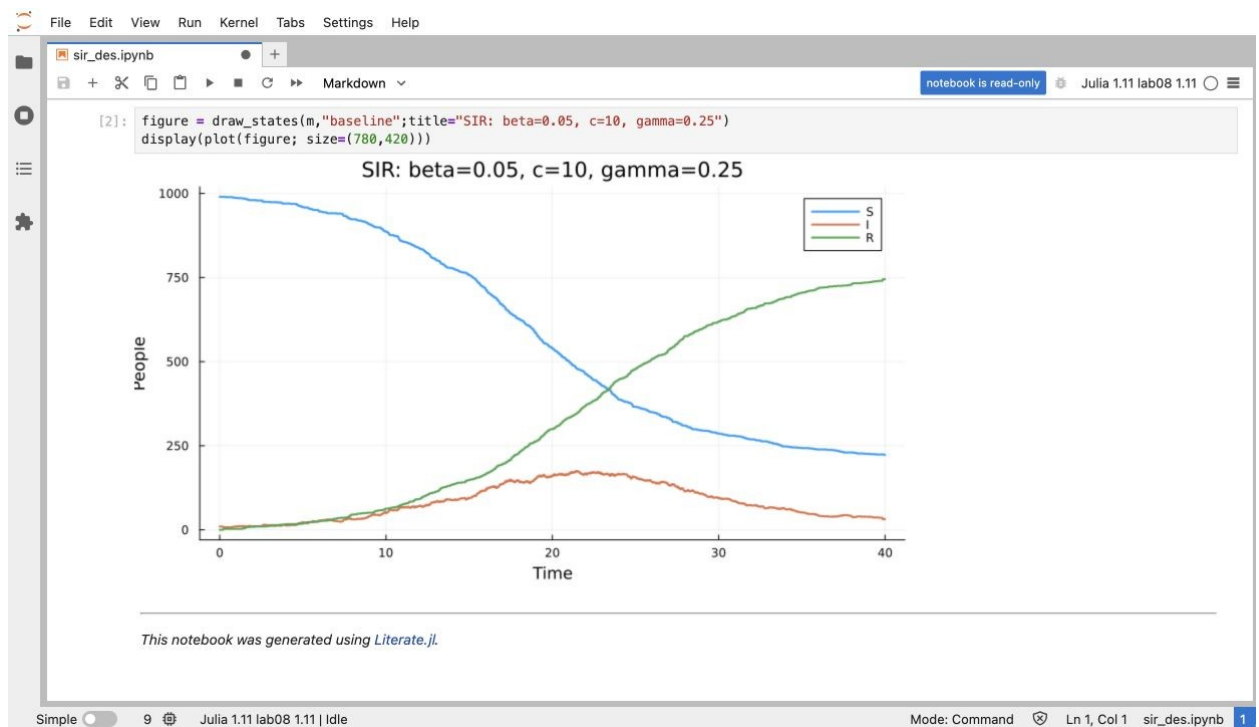


Рисунок 3: Выполненный блоком `sir_des`

Рисунок 3 представлены выполненные ячейки и результат соответствующего сценария.

4.5 Чувствительность к параметрам

Меняется один параметр за раз. По 20 повторов, горизонт 80.

```
julia --project=. scripts/sir_des_param.jl
```

4.5.1 Литературный источник

Меняется один параметр за раз. По 20 повторов, горизонт 80.

```
using DrWatson
@quickactivate "ContactEpidemics"
using ContactEpidemics.ContactProcesses
using ContactEpidemics.StudyTools
using DataFrames, CSV, Plots, Statistics

rows = NamedTuple[]
panels = []
for (variable, values) in [(:beta, [0.03, 0.05, 0.07]),
                          (:contact, [6.0, 10.0, 14.0]), (:recovery, [0.15, 0.25, 0.35])]
    p = plot(xlabel="Time", ylabel="Mean I", title=string(variable))
    for value in values
        cfg = Settings(; Dict{variable=>value}...)
        result = ensemble(cfg)
        for row in eachrow(result.metrics)
            push!(rows, (; parameter=string(variable), value,
                          NamedTuple{row}...))
        end
        save_table("scan_${variable}_${value}", result.path)
        plot!(p, result.path.t, result.path.mean_I, label=string(value))
    end
    push!(panels, p)
end
metrics = DataFrame(rows)
save_table("sensitivity_runs", metrics)
summary = combine(groupby(metrics, [:parameter, :value]),
                  :peak=>mean=>:mean_peak, :time_peak=>mean=>:mean_time_peak,
                  :R_T=>mean=>:mean_R_T, :I_T=>mean=>:mean_I_T)
save_table("sensitivity_summary", summary)
display(summary)
figure = plot(panels...; layout=(3,1), size=(1050, 1100))
savefig(figure, imagefile("sensitivity"))
display(plot(figure; size=(780, 420)))
```

This page was generated using [Literate.jl](#).

parameter	value	mean_time		mean_R_T	mean_I_T
		mean_peak	_peak		
beta	0.03	33.4	20.64	249.7	2.95
beta	0.05	174.4	18.34	800.3	0.1
beta	0.07	291.6	11.77	923.6	0
contact	6	26.45	20.2	184.6	2.3
contact	10	174.4	18.34	800.3	0.1
contact	14	295.1	12.01	927.8	0
recovery	0.15	348.6	16.62	958.1	0.15
recovery	0.25	174.4	18.34	800.3	0.1
recovery	0.35	70.35	22.88	520.4	2.1

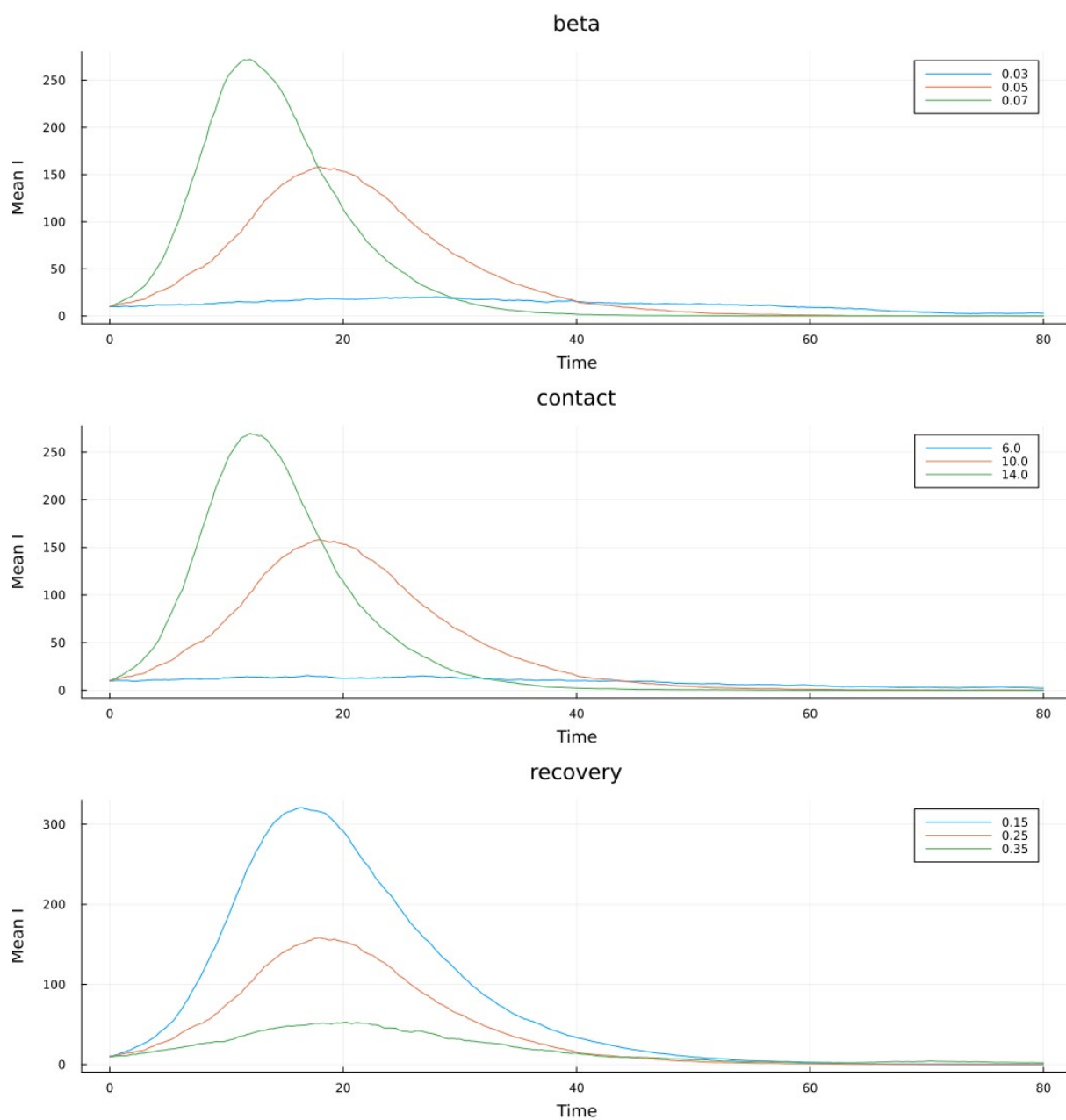


Рисунок 4: Чувствительность к параметрам: результаты расчёта

Рост β и c усиливает передачу, рост γ сокращает инфекционный период. Для сравнения используются средние метрики 20 повторов. Горизонт расширен до 80 отдельно от базового опыта.

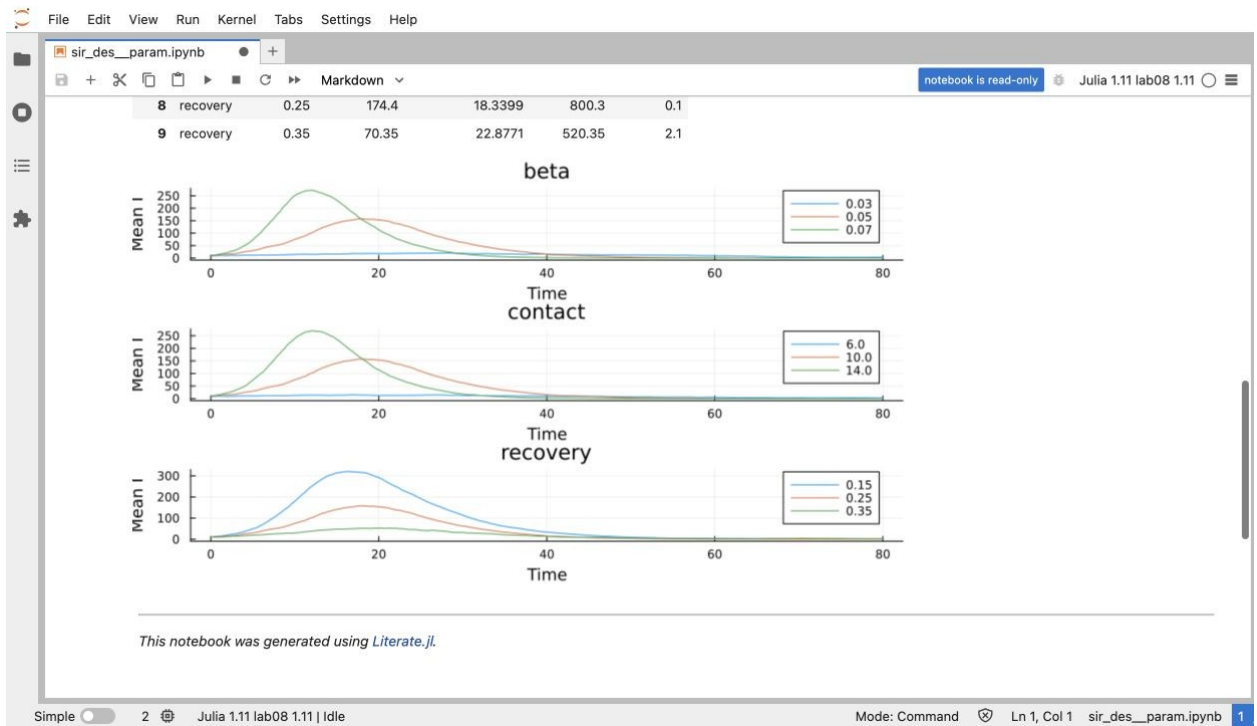


Рисунок 5: Выполненный блокнот `sir_des_param`

Рисунок 3-param представлены выполненные ячейки и результат соответствующего сценария.

4.6 Длительность инфекции

Одинаковый seed для экспоненциальной и фиксированной болезни.

```
julia --project=. scripts/sir_duration.jl
```

4.6.1 Литературный источник

Одинаковый seed для экспоненциальной и фиксированной болезни.

```
using DrWatson
@quickactivate "ContactEpidemics"
using ContactEpidemics.ContactProcesses
using ContactEpidemics.StudyTools
using DataFrames, CSV, Plots, Statistics

rows = NamedTuple{[]}(())
figure = plot(xlabel="Time", ylabel="I", title="Recovery duration")
for fixed in (false, true)
    m = simulate(config=Settings(;fixed), T=80.0)
    save_run(m; label="duration_$(fixed)", T=80.0)
    d = table(m)
    plot!(figure, d.t, d.I, label=fixed ? "fixed 4" : "exponential mean 4")
    push!(rows, (;fixed, run_metrics(m)...))
end
result = DataFrame(rows)
save_table("duration", result)
display(result)
savefig(figure, imagefile("duration"))
display(plot(figure; size=(780, 420)))
```

This page was generated using [Literate.jl](#).

fixed	peak	time_peak	R_T	I_T
false	175	21.42	815	2
true	299	12.08	794	0

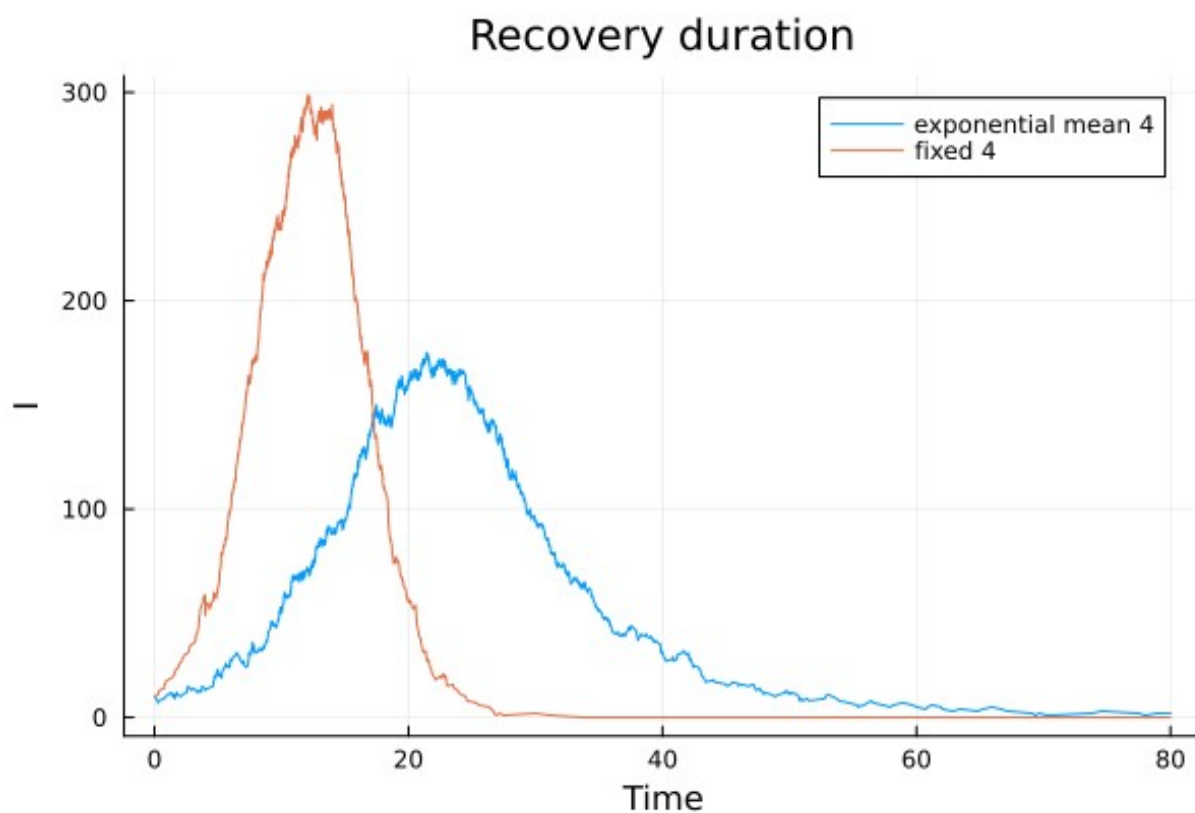


Рисунок 6: Длительность инфекции: результаты расчёта

Фиксированный период устраняет длинный хвост времени болезни. Одинаковое зерно не означает одинаковую последовательность контактов после расхождения очередей событий.

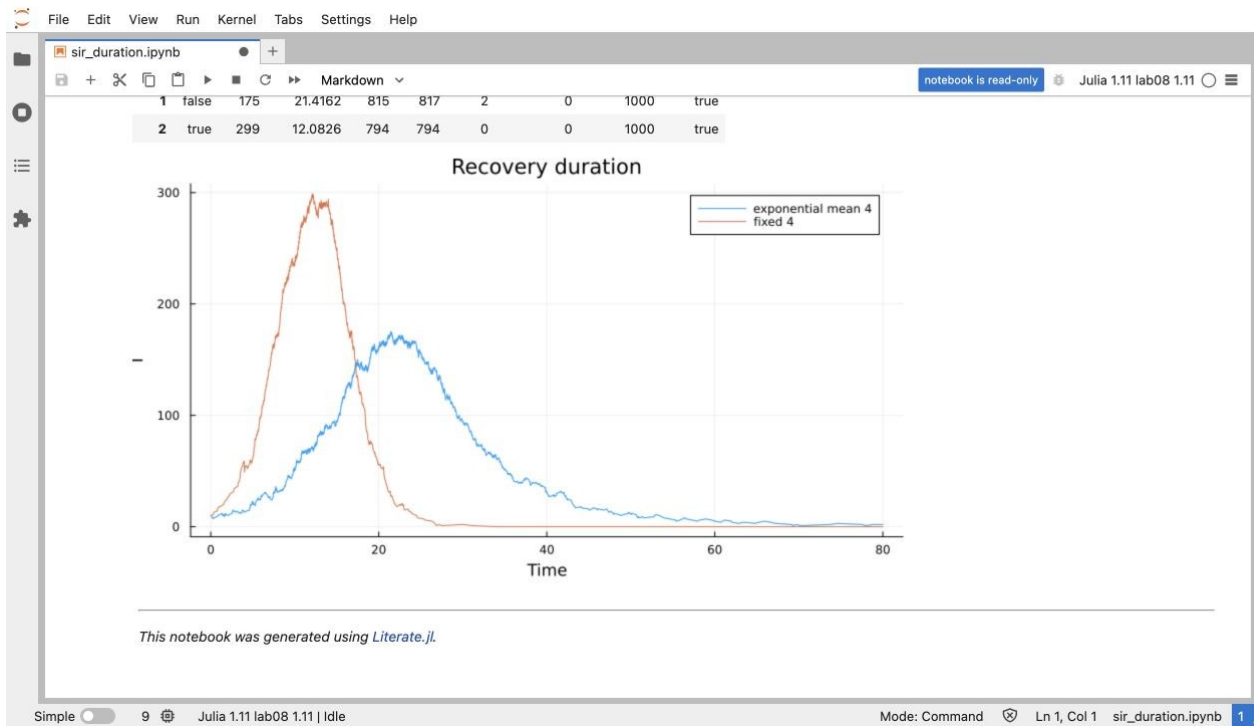


Рисунок 7: Выполненный блоком `sir_duration`

Рисунок 7 представлены выполненные ячейки и результат соответствующего сценария.

4.7 Производительность

Каждый образец использует новую активированную модель $N=10000$.

```
julia --project=. scripts/sir_benchmark.jl
```

4.7.1 Литературный источник

Каждый образец использует новую активированную модель $N=10000$.

```
using DrWatson
@quickactivate "ContactEpidemics"
using ContactEpidemics.ContactProcesses
using ContactEpidemics.StudyTools
using DataFrames, CSV, Plots, Statistics

using BenchmarkTools
fresh() = activate!(prepare(u0=(9990,10,0)))
```

Подготовка исключена из времени `sir_run`, но повторяется для каждого образца.

```
trial = @benchmark finish!(model,40.0) setup=(model=fresh()) evals=1 samples=8 seconds=120
summary = DataFrame(N=[10000],samples=[length(trial)],
    median_seconds=[median(trial).time/1e9],
    min_seconds=[minimum(trial).time/1e9],
    allocated_bytes=[median(trial).memory],
    allocations=[median(trial).allocs])
save_table("benchmark",summary)
display(trial)
display(summary)
figure = scatter(1:length(trial),trial.times/1e9,
    xlabel="Sample",ylabel="Seconds",label="fresh population",title="N=10000")
```



```
savefig(ffigure,imagefile("benchmark"))
display(plot(ffigure; size=(780,420)))
```

This page was generated using [Literatе.jl](#).

N	samples	median_secon ds	min_seconds	allocated_byt es	allocations
10000	8	2.098	1.995	1215293328	43239718

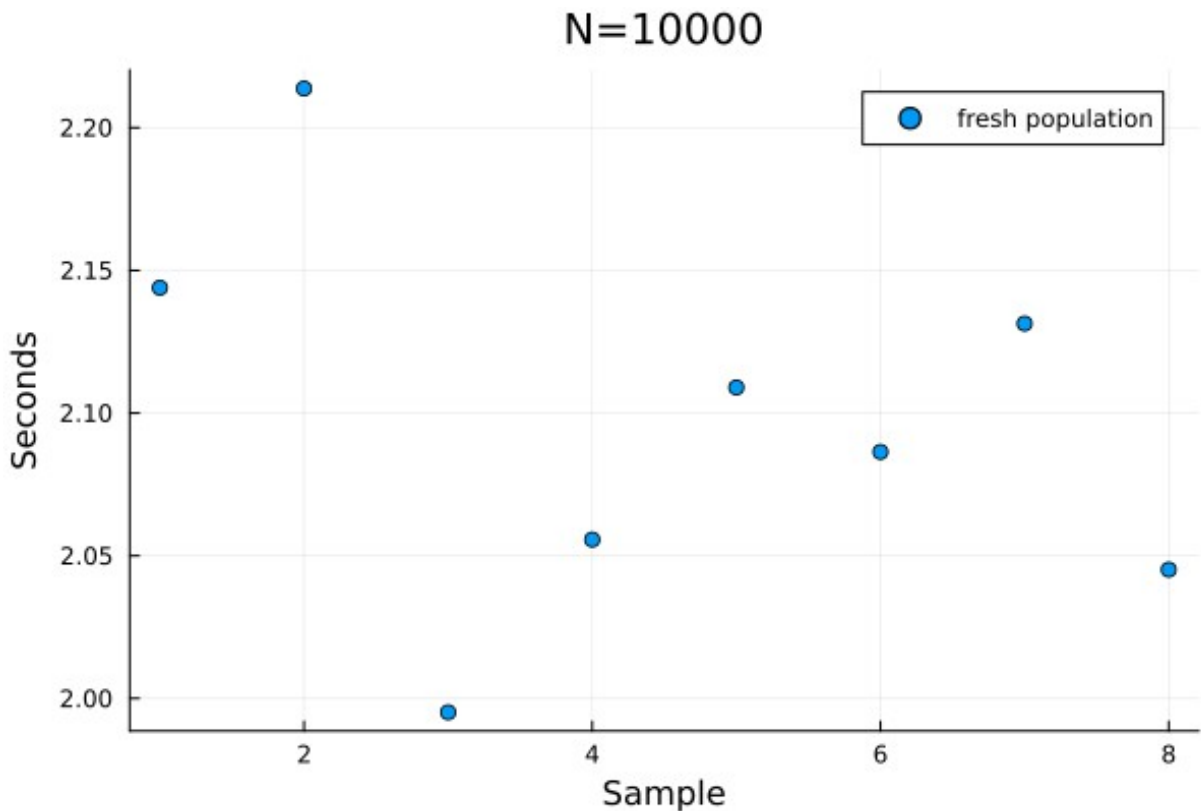


Рисунок 8: Производительность: результаты расчёта

Время измеряется только для выполнения активированной модели. setup создаёт новую модель перед каждым образцом, evals=1 исключает измерение уже завершённого процесса. Allocated bytes означает суммарные выделения, а не пиковую резидентную память. Для ускорения стоит уменьшать выделения объектов событий и стоимость контактов.

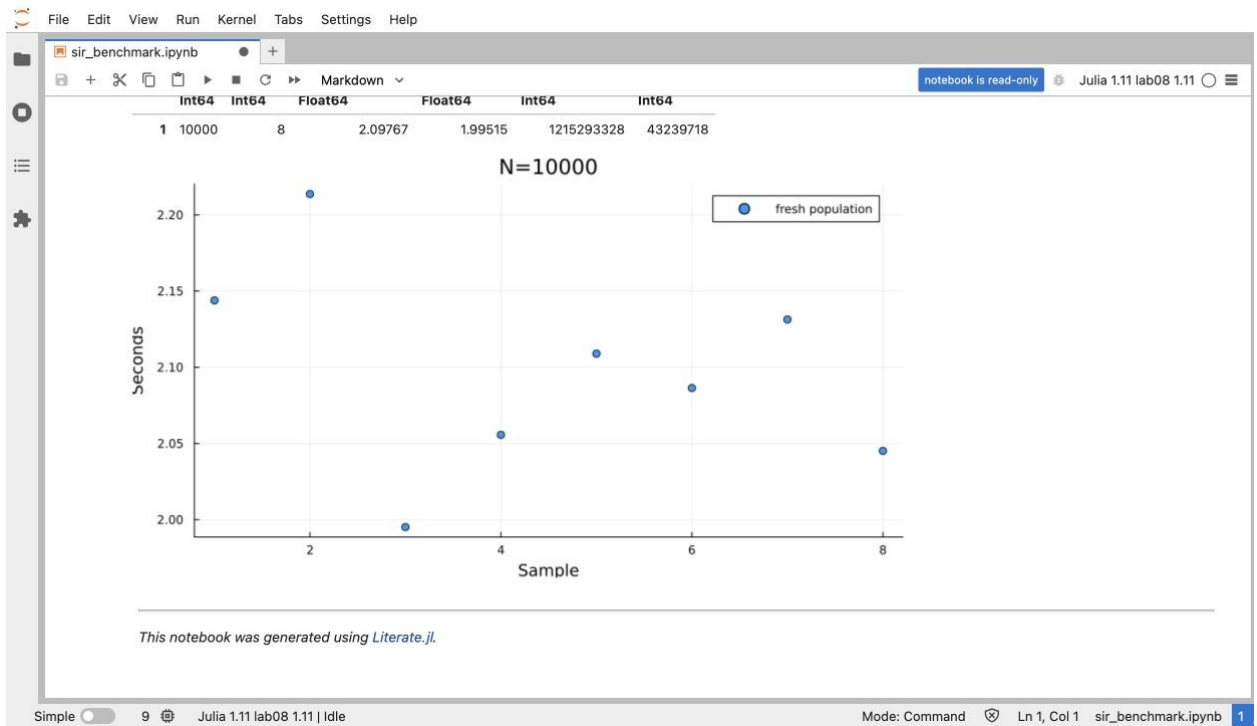


Рисунок 9: Выполненный блоком `sir_benchmark`

Рисунок 9 представлены выполненные ячейки и результат соответствующего сценария.

4.8 Анализ сохранённых CSV

Этот сценарий не запускает модель: входом служит базовый CSV.

```
julia --project=. scripts/sir_csv_report.jl
```

4.8.1 Литературный источник

Этот сценарий не запускает модель: входом служит базовый CSV.

```
using DrWatson
@quickactivate "ContactEpidemics"
using ContactEpidemics.ContactProcesses
using ContactEpidemics.StudyTools
using DataFrames, CSV, Plots, Statistics

folder = datadir("sims")
files = sort(filter(name -> startswith(name,"sir_N1000_"),readdir(folder)))
isempty(files) && error("Run sir_des.jl first")
data = CSV.read(joinpath(folder,first(files)),DataFrame)
@assert all(data.S .+ data.E .+ data.I .+ data.R .== 1000)
k = argmax(data.I)
summary = DataFrame(source=[first(files)],rows=[nrow(data)],
    peak=[data.I[k]],time_peak=[data.t[k]],R_T=[last(data.R)])
save_table("csv_review",summary)
display(summary)
figure = plot(data.t,Matrix(data[:,[:S,:I,:R]]),
    label=["S" "I" "R"],xlabel="Time",ylabel="People",
    seriestype=:steppost,title="Saved event ledger")
savefig(figure,imagefile("csv-review"))
display(plot(figure; size=(780,420)))
```

This page was generated using [Literate.jl](#).

rows	peak	time_peak	R_T
1514	175	21.42	745

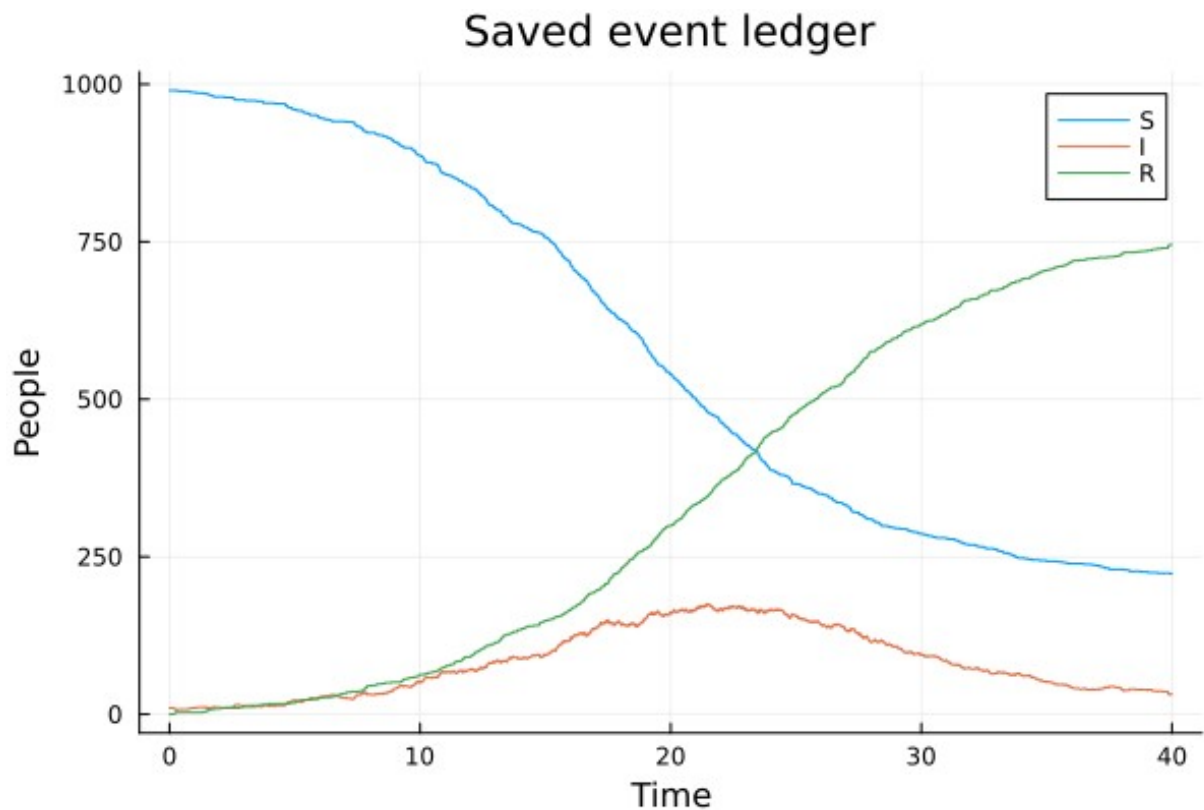


Рисунок 10: Анализ сохранённых CSV: результаты расчёта

Сценарий читает ранее созданный файл и проверяет баланс. В нём отсутствует вызов `simulate`, поэтому график является результатом анализа сохранённых данных.

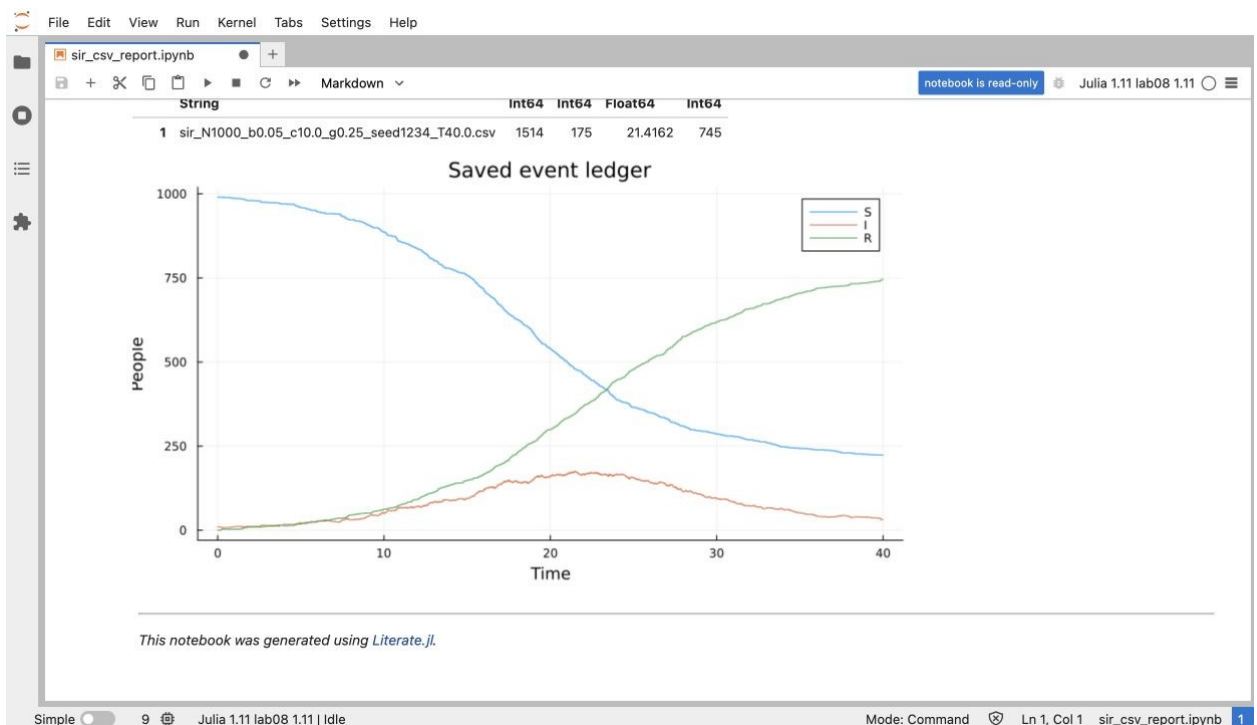


Рисунок 11: Выполненный блоком `sir_csv_report`

Рисунок 11 представлены выполненные ячейки и результат соответствующего сценария.

4.9 Рождения и смерти

Смерть независима от болезни. Поток рождений 10, смертность 0.01.

```
julia --project=. scripts/sir_demography.jl
```

4.9.1 Литературный источник

Смерть независима от болезни. Поток рождений 10, смертность 0.01.

```
using DrWatson
@quickactivate "ContactEpidemics"
using ContactEpidemics.ContactProcesses
using ContactEpidemics.StudyTools
using DataFrames, CSV, Plots, Statistics

cfg = Settings(mortality=0.01,births=10.0)
m = simulate(config=cfg,T=400.0)
@assert check_balance(m)
save_run(m;label="demography",T=400.0)
data = table(m)
summary = DataFrame([(;run_metrics(m)... ,births=last(data.B),
    deaths=last(data.D),endemic_S=1000*(0.25+0.01)/0.5,
    endemic_I=0.01*(1000-520)/(0.25+0.01))]
save_table("demography",summary)
display(summary)
figure = draw_states(m,"demography";title="Births=10, mortality=0.01")
display(plot(figure; size=(780,420)))
```

This page was generated using [Literate.jl](#).

peak	I_T	population	births	deaths
169	0	1016	4242	4226

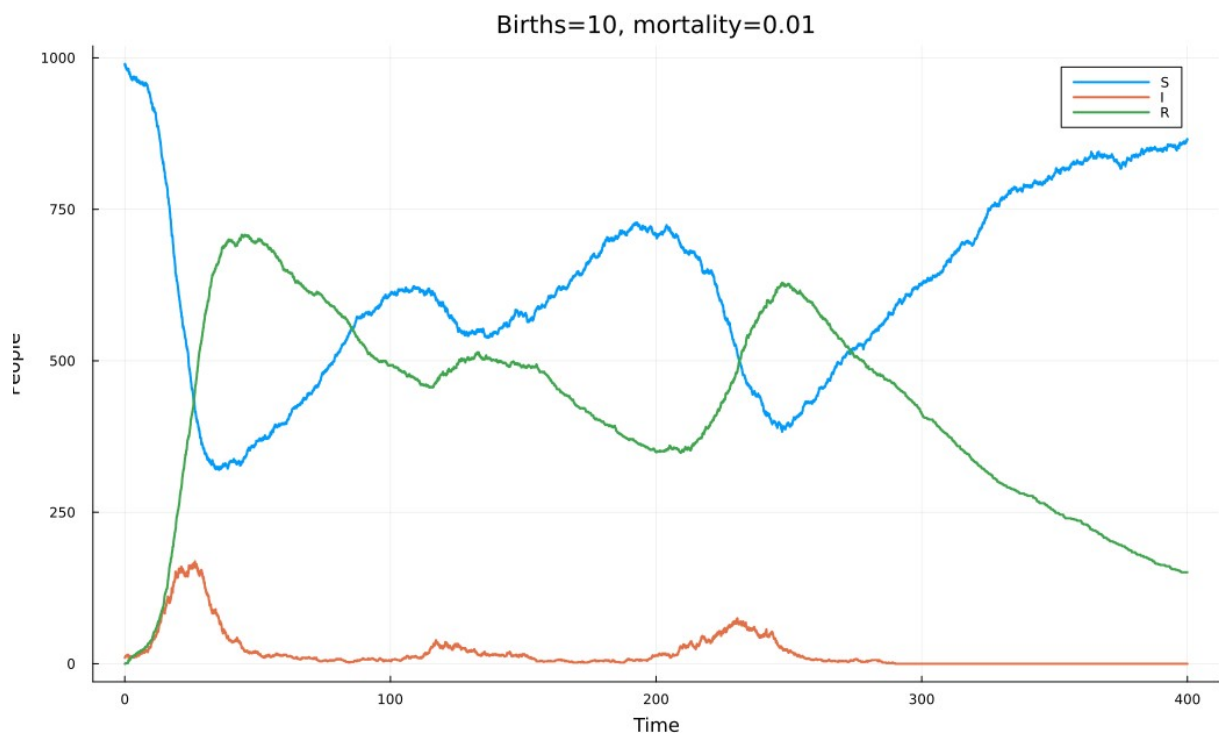


Рисунок 12: Рождения и смерти: результаты расчёта

При $\Lambda=10$ и $\mu=0.01$ средняя равновесная численность равна 1000. Для приближения с частотной передачей эндемические значения около $S=520$ и $I=18.46$. Положительное равновесие ОДУ не гарантирует сохранение инфекции в конечной популяции без заноса. Один опыт $T=400$ не является статистическим доказательством стационарности.

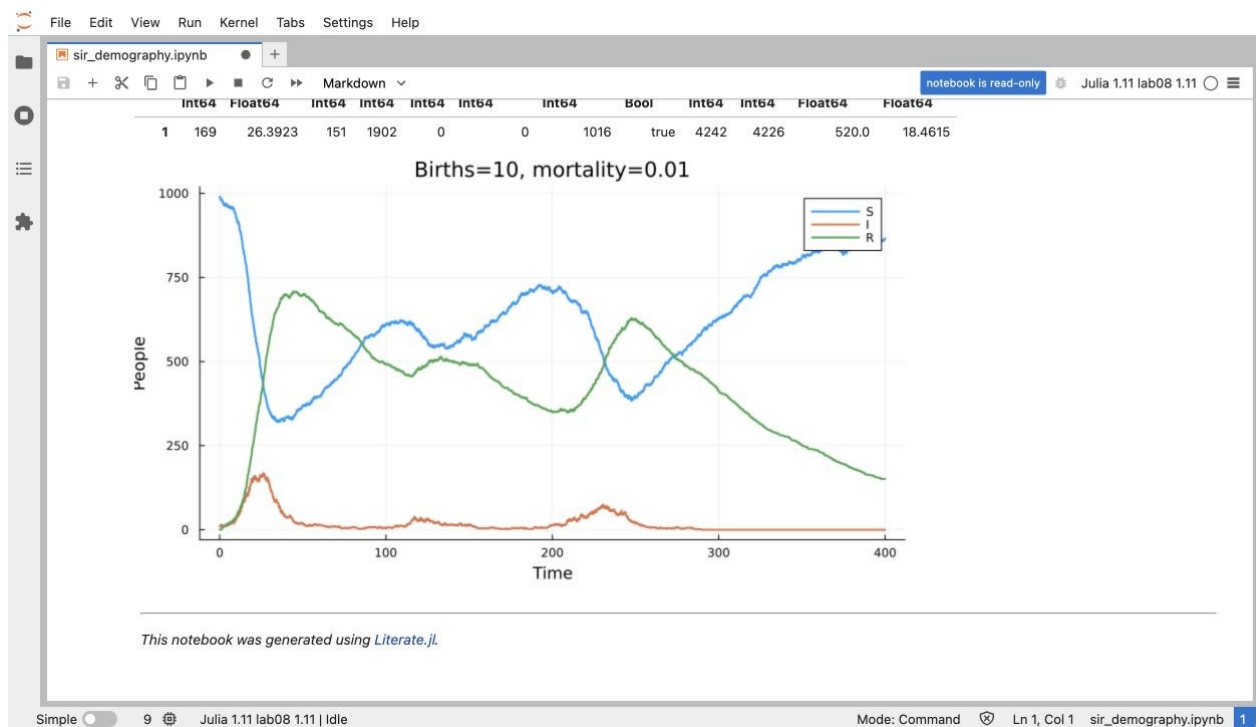


Рисунок 13: Выполненный блоком `sir_demography`

Рисунок 13 представлены выполненные ячейки и результат соответствующего сценария.

4.10 Вакцинация

Вакцинация в $t=0$, по 20 повторов для пяти долей восприимчивых.

```
julia --project=. scripts/sir_vaccination.jl
```

4.10.1 Литературный исходник

Вакцинация в $t=0$, по 20 повторов для пяти долей восприимчивых.

```
using DrWatson
@quickactivate "ContactEpidemics"
using ContactEpidemics.ContactProcesses
using ContactEpidemics.StudyTools
using DataFrames, CSV, Plots, Statistics

rows = NamedTuple{ }[]
for fraction in [0.0,0.25,0.5,0.65,0.8]
    cfg = Settings(vaccination_time=0.0,vaccination_fraction=fraction)
    result = ensemble(cfg)
    for row in eachrow(result.metrics)
        push!(rows, (;fraction,NamedTuple{ }(row)...))
    end
end
metrics = DataFrame(rows)
save_table("vaccination_runs",metrics)
summary = combine(groupby(metrics,:fraction),:peak=>mean=>:mean_peak,
    :cases=>mean=>:mean_cases,:vaccinated=>mean=>:vaccinated,
    :I_T=>mean=>:mean_I_T)
save_table("vaccination_summary",summary)
display(summary)
figure = plot(summary.fraction,summary.mean_cases,marker=:circle,
    xlabel="Vaccinated fraction of S(0)",ylabel="Cumulative infections",
    label="20 replicates",title="Vaccination at t=0")
savefig(figure,imagefile("vaccination"))
display(plot(figure; size=(780,420)))
```

This page was generated using [Literate.jl](#).

fraction	mean_peak	mean_cases	vaccinated	mean_I_T
0	179.3	795.5	0	0.15
0.25	63.2	419.1	247	1.7
0.5	18.35	98.05	495	0.3
0.65	11.55	29	643	0
0.8	10.35	14.75	792	0

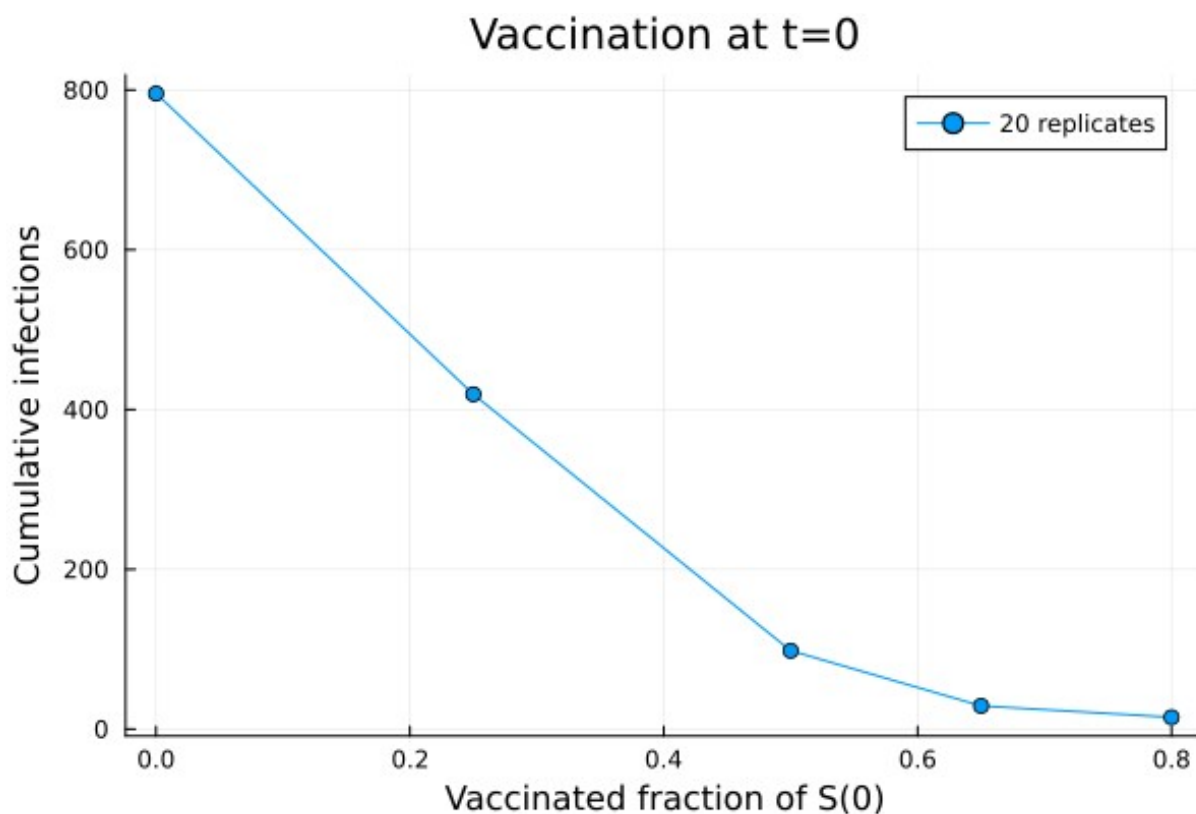


Рисунок 14: Вакцинация: результаты расчёта

Порог раннего роста около 49.55% относится к доле исходных восприимчивых. Опыты с долями 0, 0.25, 0.5, 0.65 и 0.8 показывают изменение среднего накопленного числа инфекций. Вакцинированные не включаются в cases.

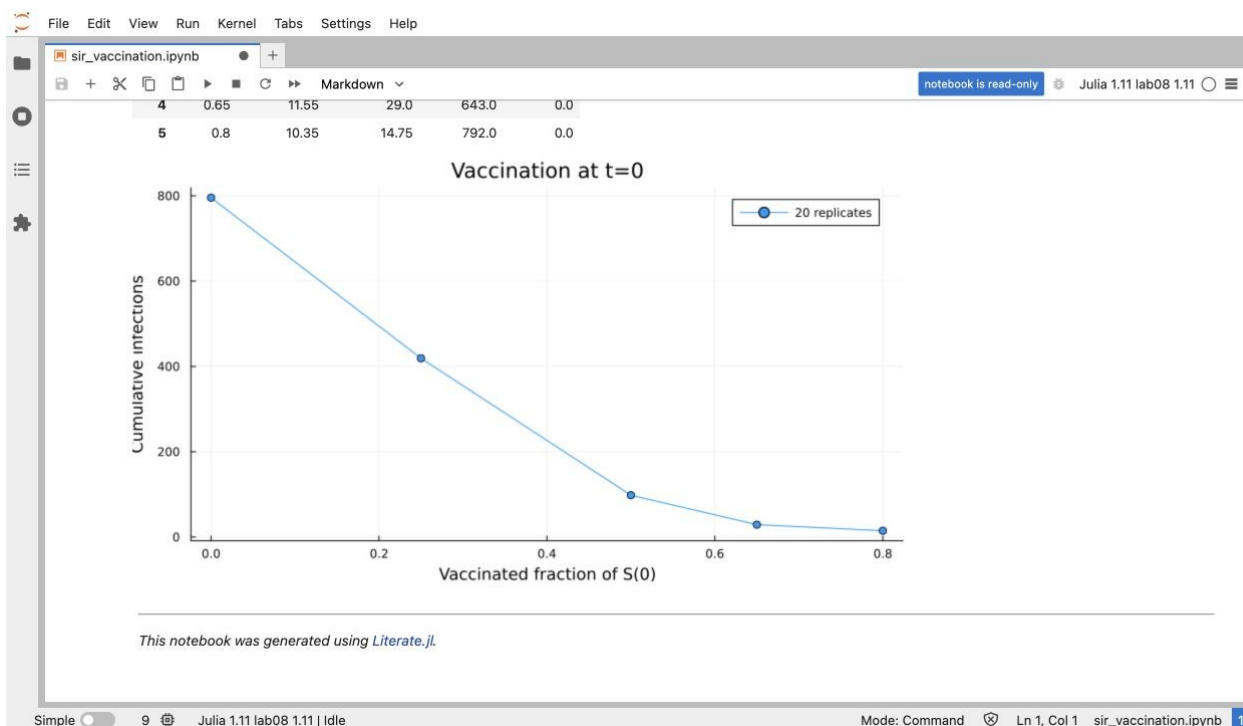


Рисунок 15: Выполненный блокнот sir_vaccination

Рисунок 15 представлены выполненные ячейки и результат соответствующего сценария.

4.11 Латентная стадия

Средний латентный период 2; сравнение с SIR при одинаковом seed.

```
julia --project=. scripts/sir_seir.jl
```

4.11.1 Литературный исходник

Средний латентный период 2; сравнение с SIR при одинаковом seed.

```
using DrWatson
@quickactivate "ContactEpidemics"
using ContactEpidemics.ContactProcesses
using ContactEpidemics.StudyTools
using DataFrames, CSV, Plots, Statistics

rows = NamedTuple[]
figure = plot(xlabel="Time",ylabel="I",title="SIR and SEIR")
for sigma in [Inf,0.5]
    m = simulate(config=Settings(latent=sigma),T=80.0)
    @assert check_balance(m)
    save_run(m;label="latent_$(sigma)",T=80.0)
    d = table(m)
    plot!(figure,d.t,d.I,label=isfinite(sigma) ? "SEIR sigma=0.5" : "SIR")
    push!(rows, (;sigma,run_metrics(m)...,peak_E=maximum(d.E)))
    isfinite(sigma) && draw_states(m,"seir-states";title="SEIR compartments")
end
summary = DataFrame(rows)
save_table("seir",summary)
display(summary)
savefig(figure,imagefile("seir"))
display(plot(figure; size=(780,420)))
```

This page was generated using [Literate.jl](#).

sigma	peak	time_peak	peak_E	R_T	I_T
inf	175	21.42	0	815	2
0.5	116	34.51	74	798	0

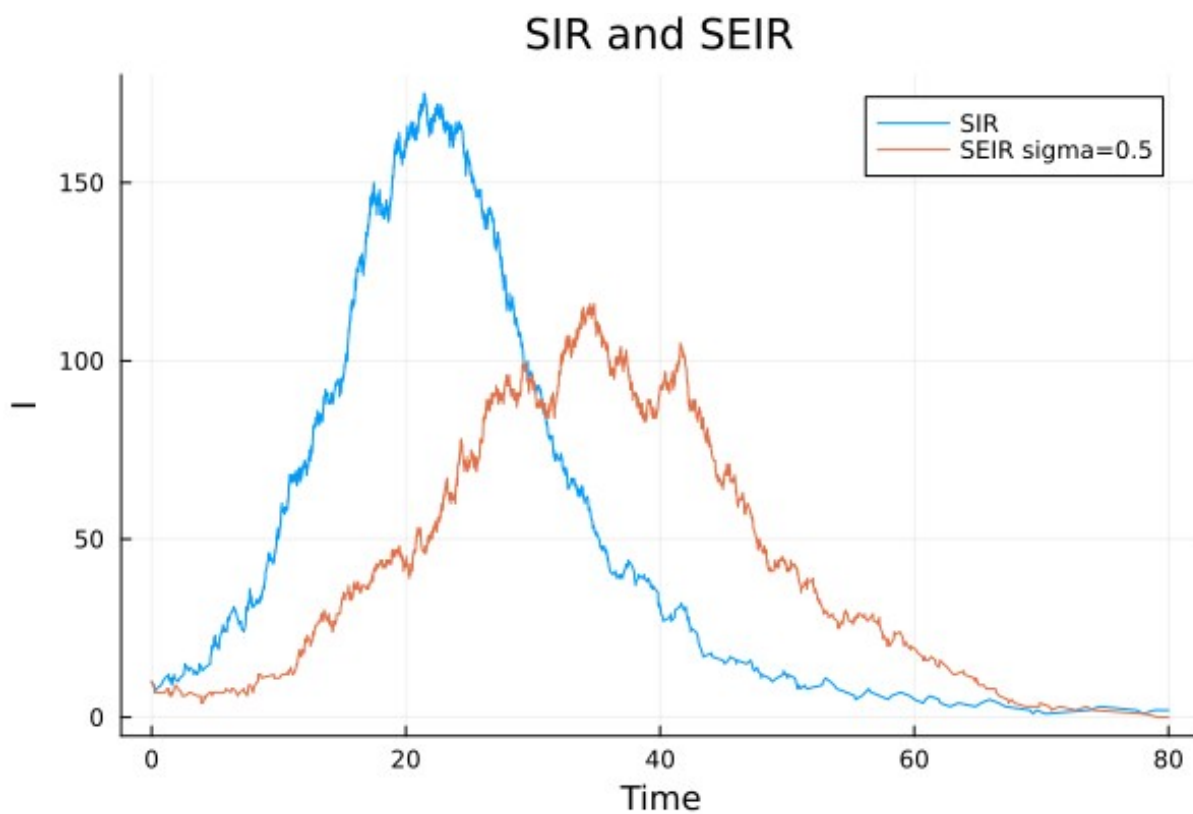


Рисунок 16: Латентная стадия: результаты расчёта

Латентная стадия откладывает появление инфекционных агентов. После смерти отложенное событие $E \rightarrow I$ отменяется проверкой состояния. Помимо сравнения I сохранены все четыре ряда $S/E/I/R$.

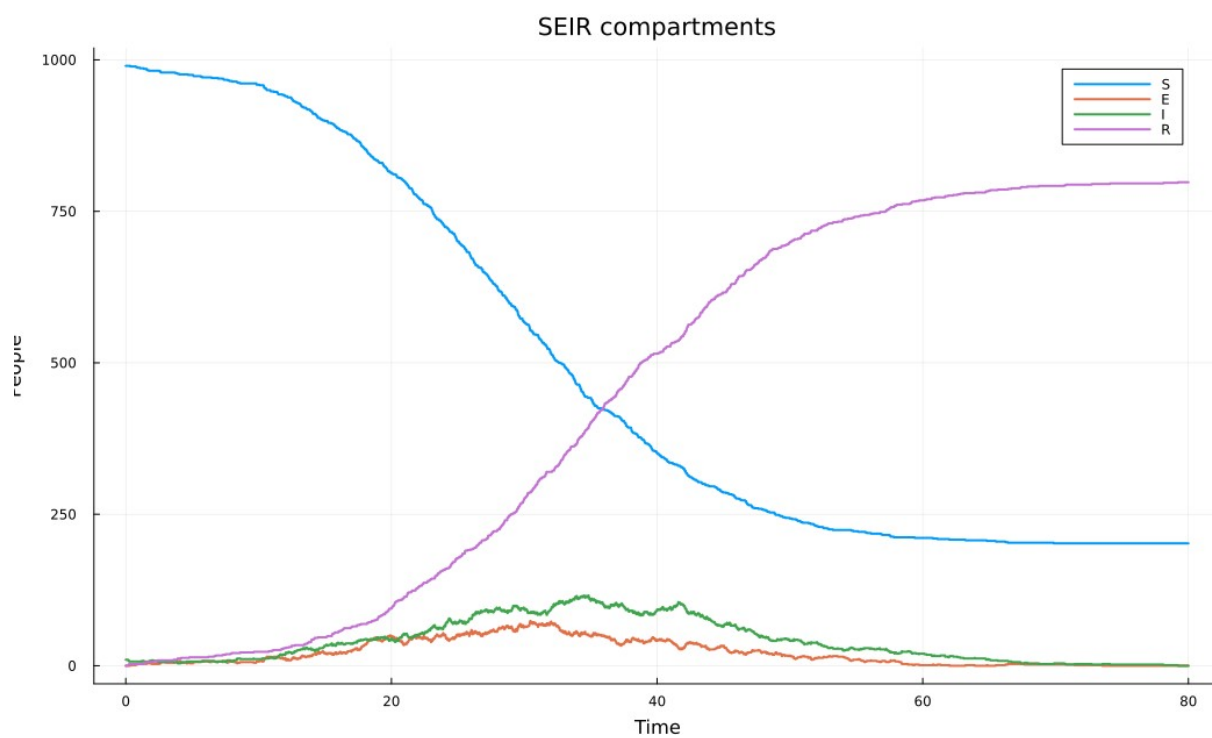


Рисунок 17: Все состояния SEIR

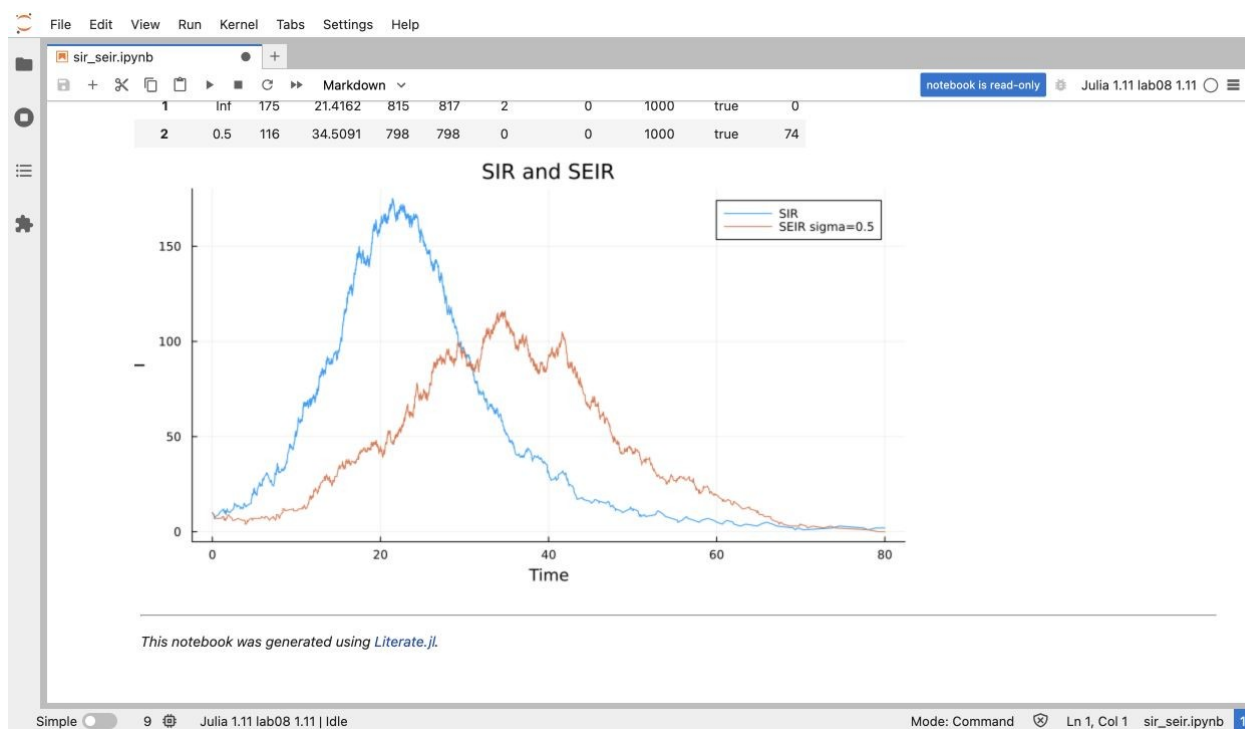


Рисунок 18: Выполненный блокнот `sir_seir`

Рисунок 18 представлены выполненные ячейки и результат соответствующего сценария.

4.12 Сопоставление с ОДУ

Средний ансамбль из 30 прогонов сопоставляется с RK4 и шагом 0.01.

```
julia --project=. scripts/sir_compare_ode.jl
```

4.12.1 Литературный исходник

Средний ансамбль из 30 прогонов сопоставляется с RK4 и шагом 0.01.

```
using DrWatson
@quickactivate "ContactEpidemics"
using ContactEpidemics.ContactProcesses
using ContactEpidemics.StudyTools
using DataFrames, CSV, Plots, Statistics

function rhs(u)
    flow = 0.05*10*u[1]*u[2]/999
    recovery = 0.25*u[2]
    [-flow, flow-recovery, recovery]
end

function integrate(h)
    u = [990.0, 10.0, 0.0]
    rows = [(t=0.0, S=u[1], I=u[2], R=u[3])]
    for step in 1:round(Int, 80/h)
        a = rhs(u)
        b = rhs(u+h*a/2)
        c = rhs(u+h*b/2)
        d = rhs(u+h*c)
        u += h*(a+2b+2c+d)/6
        step % round(Int, 0.25/h) == 0 &&
            push!(rows, (t=step*h, S=u[1], I=u[2], R=u[3]))
    end
    DataFrame(rows)
end

ode = integrate(0.01)
```

```

refined = integrate(0.005)
@assert maximum(abs.(ode.I-refined.I)) < 1e-5
@assert maximum(abs.(ode.S+ode.I+ode.R .- 1000)) < 1e-7
result = ensemble(Settings(); repeats=30)
summary = DataFrame(ode_peak=[maximum(ode.I)],
    peak_mean_path=[maximum(result.path.mean_I)],
    mean_individual_peak=[mean(result.metrics.peak)],
    rmse=[sqrt(mean((ode.I-result.path.mean_I).^2))],
    step_difference=[maximum(abs.(ode.I-refined.I))])
save_table("ode",ode)
save_table("ensemble",result.path)
save_table("ode_comparison",summary)
display(summary)
figure = plot(ode.t,ode.I,label="ODE",xlabel="Time",ylabel="I")
plot!(figure,result.path.t,result.path.mean_I,
    ribbon=result.path.sd_I,label="DES mean +/- SD",fillalpha=0.15)
savefig(figure,imagefile("ode-comparison"))
display(plot(figure; size=(780,420)))

```

This page was generated using [Literat.jl](#).

ode_peak	peak_mean_path	mean_individual_peak	rmse	step_difference
158.8	154.8	175.4	1.765	2.25e-11

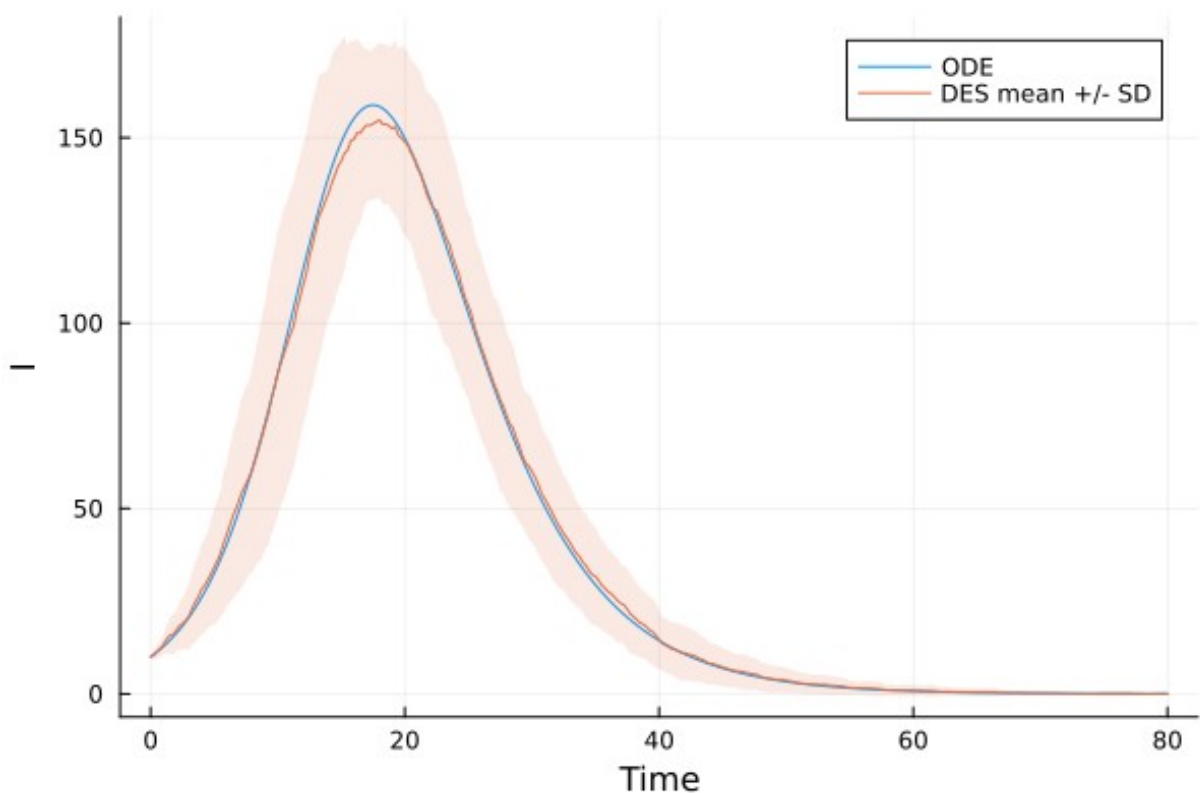


Рисунок 19: Сопоставление с ОДУ: результаты расчёта

RK4 проверяется уменьшением шага с 0.01 до 0.005. Полоса вокруг среднего DES показывает стандартное отклонение траекторий, а не доверительный интервал среднего. Максимум средней траектории и среднее индивидуальных максимумов вычисляются отдельно.

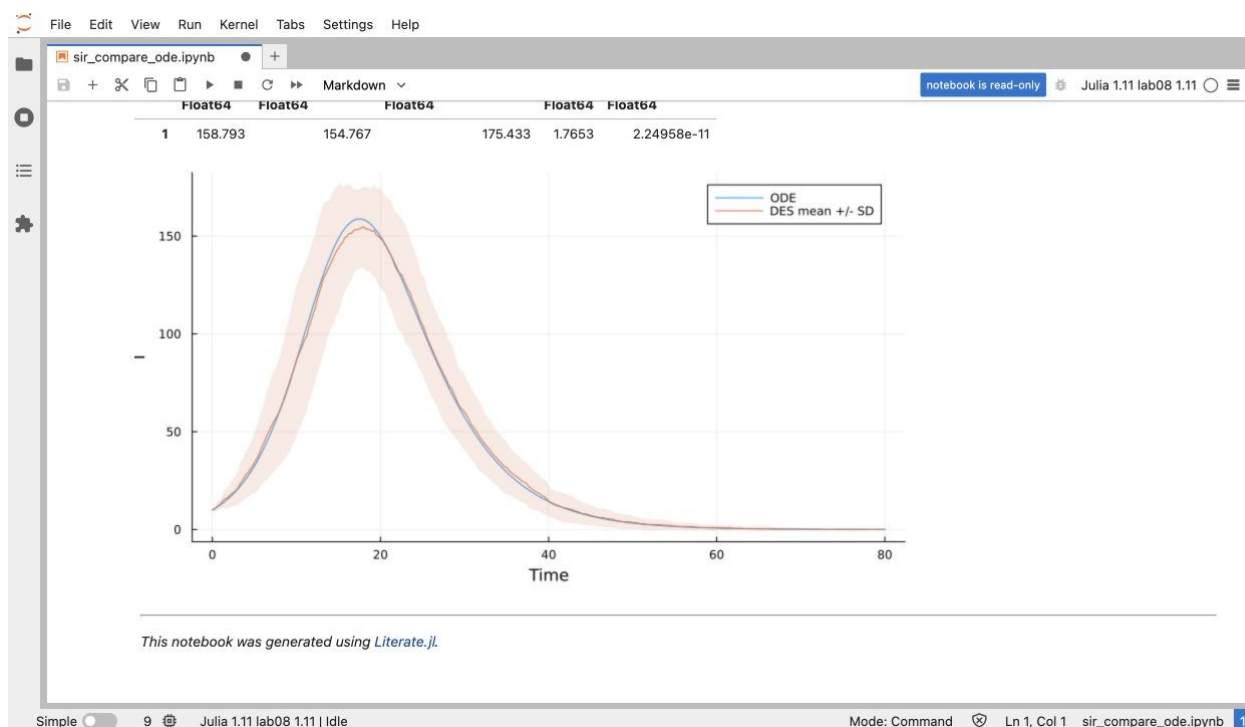


Рисунок 20: Выполненный блокнот `sir_compare_ode`

Рисунок 20 представлены выполненные ячейки и результат соответствующего сценария.

4.13 Производные форматы

Literate преобразует один литературный источник в чистый Julia-скрипт, Jupyter notebook и Quarto QMD [4]. Каждый из девяти блокнотов выполняется, включая параметры, benchmark и чтение CSV. В тематические разделы этого отчёта подключены полные QMD-фрагменты, сформированные генератором. Повторные копии исходников в виде чистых скриптов не печатаются второй раз.

```
make notebooks
make docs
```

Листинг файла `project/scripts/tangle.jl`.

```
using Pkg
Pkg.activate(joinpath(@__DIR__, ".."))
using Literate
root = dirname(@__DIR__)
execute = "--execute" in ARGS
names = ["sir_des", "sir_des_param", "sir_duration", "sir_benchmark",
         "sir_csv_report", "sir_demography", "sir_vaccination", "sir_seir",
         "sir_compare_ode"]
for name in names
    source = joinpath(@__DIR__, name*".jl")
    clean = joinpath(root, "generated", name)
    nb = joinpath(root, "notebooks", name)
    docs = joinpath(root, "markdown", name)
    Literate.script(source, clean; documenter=false)
    Literate.notebook(source, nb; execute=documenter=false)
    Literate.markdown(source, docs; flavor=Literate.QuartoFlavor())
end
```

```

qmd = joinpath(docs,name*".qmd")
# The full generated prose and code are included without rerunning in Word.
text = replace(read(qmd,String),"``{julia}"=>"``julia")
text = replace(text,r"(?m)^(# [^\n]+)"=>"### Литературный источник")
write(joinpath(docs,name*"-report.qmd"),text)
end

```

4.14 Проверки модели и сборка

Проверяются баланс на каждом событии, тождественность повторов с одинаковым seed, отсутствие передачи при $\beta=0$, точное выздоровление при фиксированном периоде, случаи $N=0$ и $N=1$, отсутствие действий умершего агента, полная вакцинация, латентная стадия, демография и правостороннее чтение временного ряда.

```

rhktrlwmd@Mac % make test
Test Summary: | Pass Total Time
Conservation and reproducibility | 5 5 1.1s
Test Summary: | Pass Total Time
No transmission and deterministic recovery | 4 4 0.1s
Test Summary: | Pass Total Time
Competing processes cannot resurrect people | 27 27 0.0s
Test Summary: | Pass Total Time
Vaccination and latent infection | 5 5 0.3s
Test Summary: | Pass Total Time
Demographic balance and sampling | 5 5 0.3s
rhktrlwmd@Mac %

```

Рисунок 21: Результат тестов

Рисунок 21 содержит фактический вывод набора тестов.

Листинг файла project/test/runtests.jl.

```

using Test, ContactEpidemics.ContactProcesses
@testset "Conservation and reproducibility" begin
    m = simulate()
    @test check_balance(m)
    @test table(m) == table(simulate())
    @test first(table(m).I) == 10
    @test last(table(m).t) == 40.0
    @test all(diff(table(m).S) .<= 0)
end
@testset "No transmission and deterministic recovery" begin
    m = simulate(config=Settings(beta=0.0, fixed=true), T=5.0)
    @test m.counts == [990,0,0,10]
    @test all(row.t == 4.0 for row in m.log if row.event == :recovery)
    @test simulate(u0=(1,0,0)).counts == [1,0,0,0]
    @test simulate(u0=(0,0,0)).counts == [0,0,0,0]
end
@testset "Competing processes cannot resurrect people" begin

```

```

m = simulate(u0=(10,10,0),T=30.0,
             config=Settings(mortality=5.0,latent=0.5))
@test check_balance(m)
@test isempty(m.living)
dead = Set{Int}()
for row in m.log
    @test !(row.person in dead)
    row.event == :death && push!(dead,row.person)
end
@test !transition!(m,1,4,:recovery)
end
@testset "Vaccination and latent infection" begin
    m = simulate(config=Settings(vaccination_time=0.0,
                                vaccination_fraction=1.0),T=80.0)
    @test m.cases == 10
    @test m.vaccinated == 990
    @test check_balance(m)
    seir = simulate(config=Settings(latent=0.5),T=80.0)
    @test maximum(table(seir).E) > 0
    @test check_balance(seir)
end
@testset "Demographic balance and sampling" begin
    m = simulate(config=Settings(mortality=0.05,births=50.0))
    @test check_balance(m)
    @test m.born > 0 && m.died > 0
    d = table(m)
    @test sample_path(d,[0.0,40.0]) == [10,last(d.I)]
    @test_throws ArgumentError prepare(config=Settings(beta=1.1))
    @test_throws ArgumentError activate!(m)
end

```

Листинг файла project/Makefile.

```

JULIA ?= julia
SCENARIOS = sir_des sir_des__param sir_duration sir_benchmark \
             sir_csv_report sir_demography sir_vaccination \
             sir_seir sir_compare_ode
.PHONY: setup run tangle notebooks docs test all
setup:
    $(JULIA) --project=. scripts/setup_environment.jl
run:
    @for name in $(SCENARIOS); do $(JULIA) --project=. scripts/$$name.jl || exit $$?; done
tangle:
    $(JULIA) --project=. scripts/tangle.jl
notebooks:
    $(JULIA) --project=. scripts/tangle.jl --execute
docs:
    @for name in $(SCENARIOS); do \
        (cd notebooks/$$name && quarto render $$name.ipynb --to html \
         --embed-resources --no-execute --output $$name.html) || exit $$?; \
        mv notebooks/$$name/$$name.html markdown/$$name/; \
    done
test:
    $(JULIA) --project=. test/runtests.jl
all: setup run notebooks docs test

```

5. Выводы

Реализована дискретно-событийная модель с индивидуальными контактами и отдельными событиями выздоровления. Базовые условия сохранены, дополнительные опыты охватывают все пункты §8.5. Параметры передачи и длительность болезни влияют на высоту и время пика, вакцинация уменьшает накопленные инфекции, а латентная стадия меняет временной профиль эпидемии. Для демографии соблюдается баланс $S+E+I+R=N_0+B-D$.

Сравнение с ОДУ интерпретируется на уровне ансамбля. Отдельные траектории могут отклоняться от среднего, а ненулевое $I(T)$ требует осторожности при трактовке конечной доли переболевших. Из литературных источников получены и выполнены производные форматы, а их полные листинги включены в соответствующие разделы.

Список литературы

1. Королькова А. В., Кулябов Д. С. Имитационное моделирование. Практикум. Москва, 2026.
2. JuliaDynamics. ConcurrentSim Documentation [Электронный ресурс]. URL: <https://juliadynamics.github.io/ConcurrentSim.jl/stable/> (дата обращения: 08.09.2026).
3. Keeling M. J., Rohani P. Modeling Infectious Diseases in Humans and Animals. Princeton University Press, 2008.
4. Krekel F. Literate.jl Documentation [Электронный ресурс]. URL: <https://fredriekre.github.io/Literate.jl/v2/> (дата обращения: 08.09.2026).